

NC State University
Department of Electrical and Computer Engineering
ECE 463/563 (Prof. Rotenberg)
Project #1: Cache Design, Memory Hierarchy Design
REPORT TEMPLATE (Version 2.0)

by

Austin Dolan

NCSU Honor Pledge: "I have neither given nor received unauthorized aid on this project."

Student's electronic signature: **Austin Dolan**
(sign by typing your name)

Course number: **463**
(463 or 563 ?)

1. L1 cache exploration: SIZE and ASSOC

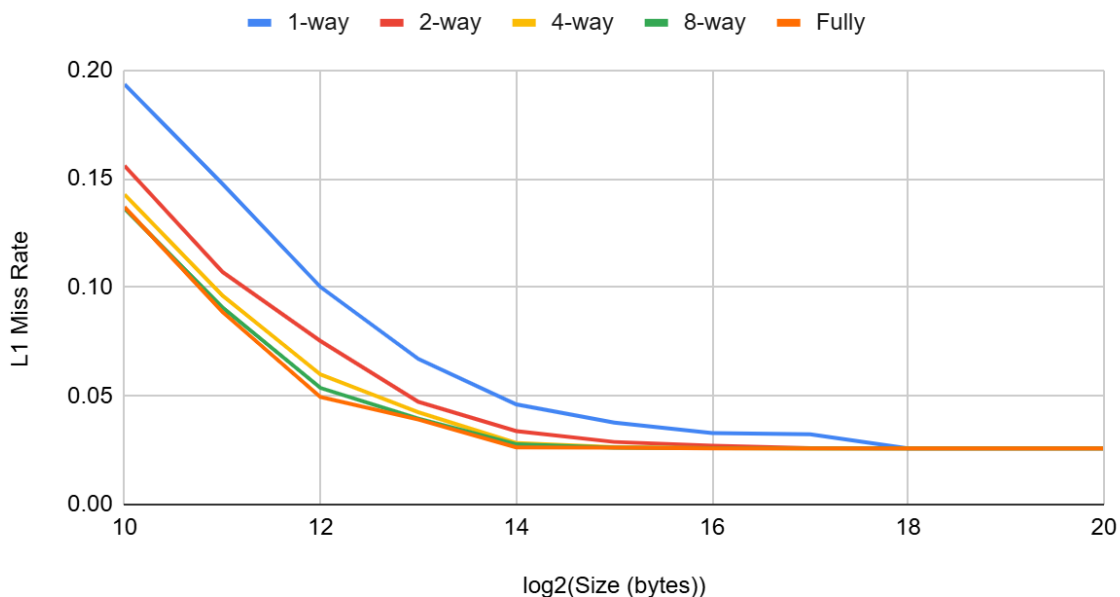
GRAPH #1 (total number of simulations: 55)

For this experiment:

- Benchmark trace: gcc_trace.txt
- L1 cache: SIZE is varied, ASSOC is varied, BLOCKSIZE = 32.
- L2 cache: None.
- Prefetching: None.

Plot L1 miss rate on the y-axis versus $\log_2(\text{SIZE})$ on the x-axis, for eleven different cache sizes: SIZE = 1KB, 2KB, ..., 1MB, in powers-of-two. (That is, $\log_2(\text{SIZE}) = 10, 11, \dots, 20$.) The graph should contain five separate curves (*i.e.*, lines connecting points), one for each of the following associativities: direct-mapped, 2-way set-associative, 4-way set-associative, 8-way set-associative, and fully-associative. All points for direct-mapped caches should be connected with a line, all points for 2-way set-associative caches should be connected with a line, *etc.*

L1 Miss Rate vs Cache Size for varying Associativities



Answer the following questions:

1. For a given associativity, how does increasing cache size affect miss rate?

Increasing the cache size decreases the miss rate for a given associativity.

2. For a given cache size, how does increasing associativity affect miss rate?

Increasing the associativity decreases the miss rate for a given cache size.

3. Estimate the *compulsory miss rate* from the graph and briefly explain how you arrived at this estimate.

compulsory miss rate = 0.025

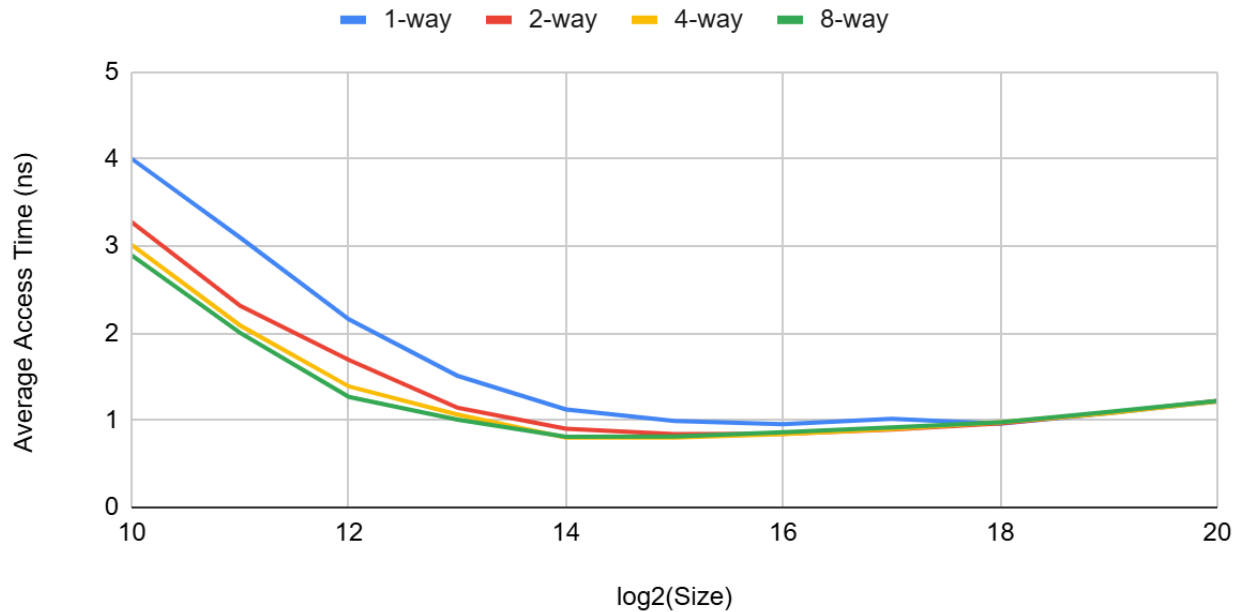
How I arrived at this estimate: The compulsory miss rate is equal to the total miss rate minus the conflict and capacity miss rates. A fully associative cache has no conflict misses, and an infinitely large cache has no capacity misses. The fully associative cache approaches 0.025 as the cache size increases, thus indicating that the compulsory miss is 0.025.

GRAPH #2 (no additional simulations with respect to GRAPH #1)

Same as GRAPH #1, except make the following changes:

- The y-axis should be AAT instead of L1 miss rate.
- Do NOT include fully-associative cache configurations (power-hungry, impractical cost for implementing LRU). Thus, GRAPH #2 should contain four (not five) separate curves: direct-mapped, 2-way set-associative, 4-way set-associative, 8-way set-associative.

Average Access Time (AAT) vs Cache Size for varying Associativity



Answer the following question:

1. For a memory hierarchy with only an L1 cache and BLOCKSIZE = 32, which configuration yields the best (*i.e.*, lowest) AAT and what is that AAT?

Direct-mapped or set-associative cache configuration that yields the lowest AAT:

<32> KB, << 4-way set-associative>>

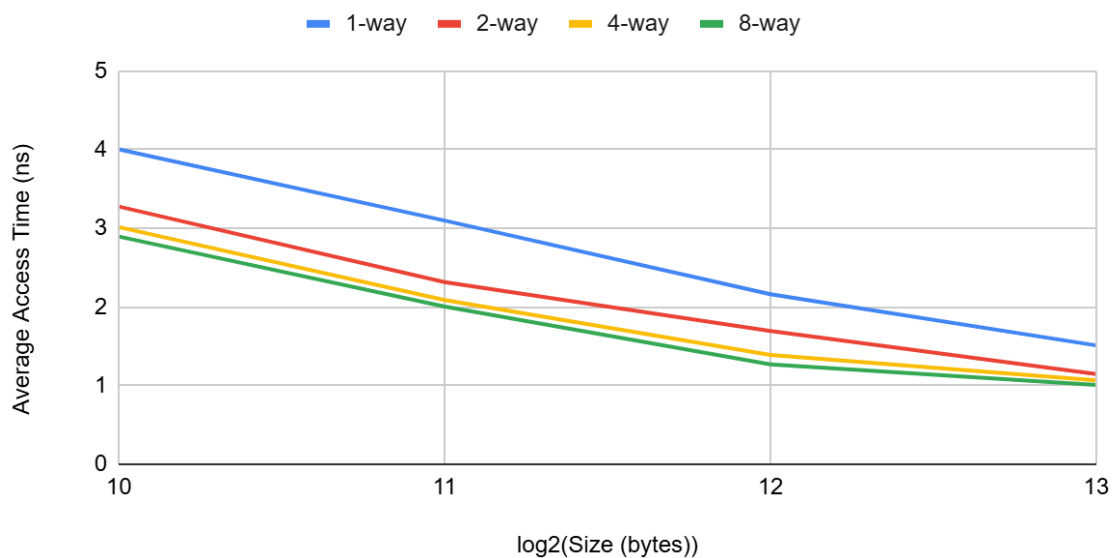
AAT for this configuration: 0.80189 (ns)

GRAPH #3 (total number of simulations: 16)

Same as GRAPH #2, except make the following changes:

- Add the following L2 cache to the memory hierarchy: 16KB, 8-way set-associative, same block size as L1 cache.
- Vary the L1 cache size only between 1KB and 8KB (since L2 cache is 16KB). (And as with GRAPH #2, GRAPH #3 should contain four (not five) separate curves for L1 associativity: direct-mapped, 2-way set-associative, 4-way set-associative, 8-way set-associative.)

Average Access Time (AAT) vs Cache Size for varying Associativity with L2 cache



Answer the following questions:

1. With the L2 cache added to the system, which L1 cache configuration yields the best (*i.e.*, lowest) AAT and what is that AAT?

L1 configuration that yields the lowest AAT with 16KB 8-way L2 added:

<<8>> KB, << 8-way set-assoc. >>

AAT for this configuration: 1.006861 (ns)

2. How does the lowest AAT with L2 cache (GRAPH #3) compare with the lowest AAT without L2 cache (GRAPH #2)?

The lowest AAT with L2 cache is << greater than >> the lowest AAT without L2 cache.

2. L1 cache exploration: SIZE and BLOCKSIZE

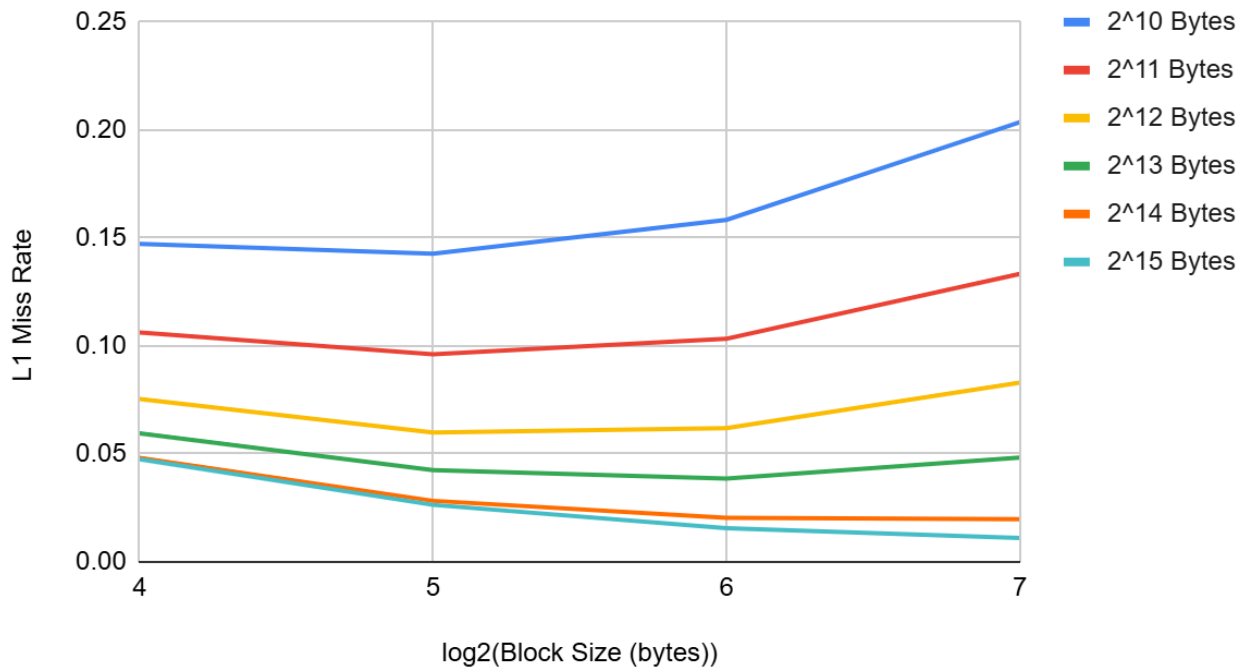
GRAPH #4 (total number of simulations: 24)

For this experiment:

- Benchmark trace: gcc_trace.txt
- L1 cache: SIZE is varied, BLOCKSIZE is varied, ASSOC = 4.
- L2 cache: None.
- Prefetching: None

Plot L1 miss rate on the y-axis versus $\log_2(\text{BLOCKSIZE})$ on the x-axis, for four different block sizes: $\text{BLOCKSIZE} = 16, 32, 64, \text{ and } 128$. (That is, $\log_2(\text{BLOCKSIZE}) = 4, 5, 6, \text{ and } 7$.) The graph should contain six separate curves (*i.e.*, lines connecting points), one for each of the following L1 cache sizes: $\text{SIZE} = 1\text{KB}, 2\text{KB}, \dots, 32\text{KB}$, in powers-of-two. All points for $\text{SIZE} = 1\text{KB}$ should be connected with a line, all points for $\text{SIZE} = 2\text{KB}$ should be connected with a line, *etc.*

L1 Miss Rate vs $\log_2(\text{Block Size})$ for varying capacity



Answer the following questions:

1. Do smaller caches prefer smaller or larger block sizes?

Smaller caches prefer << smaller >> block sizes. For example, the smallest cache considered in Graph #4 (1KB) achieves its lowest miss rate with a block size of 32 B.

2. Do larger caches prefer smaller or larger block sizes?

Larger caches prefer << larger >> block sizes. For example, the largest cache considered in Graph #4 (32KB) achieves its lowest miss rate with a block size of 128 B.

3. As block size is increased from 16 to 128, is the tension between *exploiting more spatial locality* and *cache pollution* evident in the graph? Explain.

Yes, the tension between *exploiting more spatial locality* and *cache pollution* is evident in the graph.

For example, consider the smallest (1KB) cache in Graph #4. Increasing block size from 16 B to 32 B is helpful (reduces miss rate) due to << exploiting more spatial locality >>. But then increasing block size further, from 32 B to 64 B, is not helpful (increases miss rate) due to << cache pollution >> having greater effect.

3. L1 + L2 co-exploration

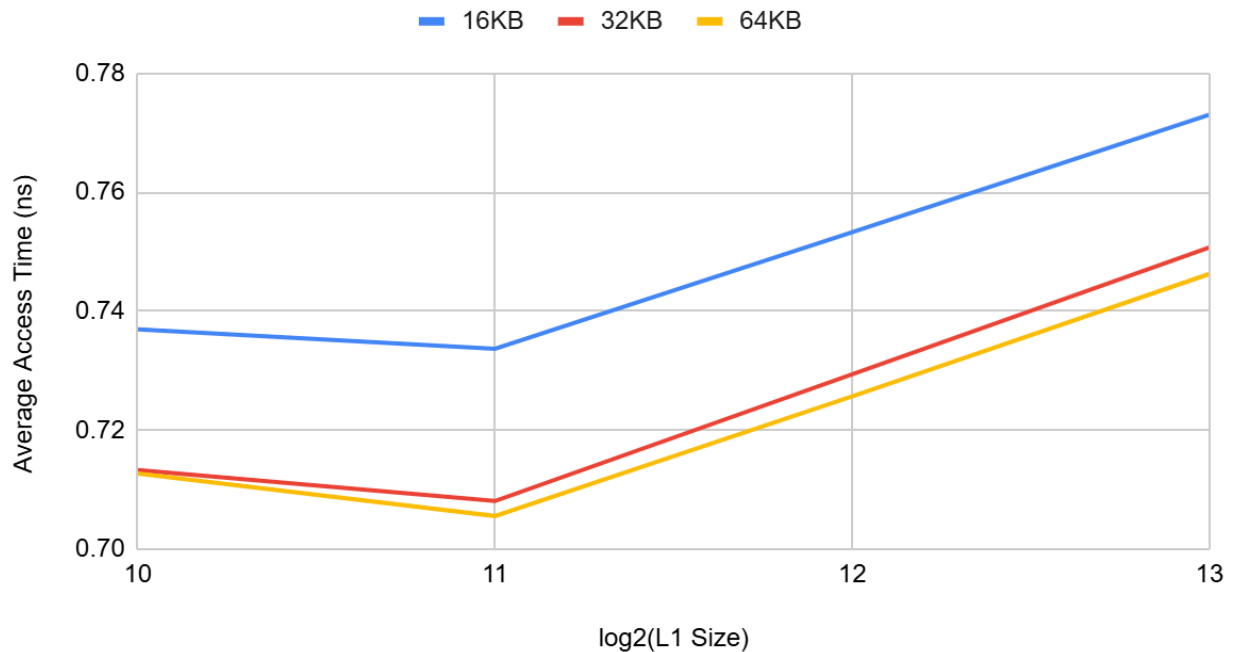
GRAPH #5 (total number of simulations: 12)

For this experiment:

- Benchmark trace: gcc_trace.txt
- L1 cache: SIZE is varied, BLOCKSIZE = 32, ASSOC = 4.
- L2 cache: SIZE is varied, BLOCKSIZE = 32, ASSOC = 8.
- Prefetching: None.

Plot AAT on the y-axis versus $\log_2(\text{L1 SIZE})$ on the x-axis, for four different L1 cache sizes: L1 SIZE = 1KB, 2KB, 4KB, 8KB. (That is, $\log_2(\text{L1 SIZE}) = 10, 11, 12, 13$.) The graph should contain three separate curves (*i.e.*, lines connecting points), one for each of the following L2 cache sizes: 16KB, 32KB, 64KB. All points for the 16KB L2 cache should be connected with a line, all points for the 32KB L2 cache should be connected with a line, *etc.*

Average Access Time (AAT) vs L1 Size for varying L2 Size



Answer the following question:

1. Which memory hierarchy configuration in Graph #5 yields the best (*i.e.*, lowest) AAT and what is that AAT?

Configuration that yields the lowest AAT: 2KB L1 and 64KB L2

Lowest AAT: 0.7055672142 (ns)